Deploy Guide — UI Only (no local CLI)

This is an alternative to DEPLOY.md for users who can't install the Databricks CLI or PyYAML locally. Everything is done inside the Databricks workspace UI.

It's slower than the one-command CLI deploy (15–25 min vs 2 min), but nothing leaves the workspace.

Two UI paths are documented:

- **A. Git Folder + workspace terminal** (recommended) — clone this repo into a Databricks Git Folder, then run `deploy.py` from a workspace terminal. Fastest UI option.
- **B. Fully manual UI** — import notebooks, create schema in SQL editor, build jobs in Workflows UI, import the dashboard JSON. No terminal needed.

Prerequisites (both paths)

- Workspace access as a user who can:
- Create a UC schema in an existing catalog (`CREATE SCHEMA` privilege)
- Create jobs (Workflows → Create Job)
- Create / edit Lakeview dashboards (Dashboards → Create)
- Read `system.billing.` *and* *`system.compute.`*
- A serverless SQL warehouse available
- The target catalog already exists
- System tables are enabled (`system.billing.usage` , `system.billing.list_prices` , `system.compute.clusters` , `system.compute.node_types` , `system.compute.node_timeline`)

Path A — Git Folder + workspace terminal (recommended)

A1. Put the project into the workspace as a Git Folder

1. Push this folder to a Git repo you can access from the workspace (GitHub / GitLab / Bitbucket / Azure DevOps).
2. In the workspace UI: **Workspace** → **Repos** → **Add Repo** → **clone URL**.
3. Pick your branch; folder now appears under `/Workspace/Users/<you>/gcp-cost-observability` (or wherever you cloned it).

A2. Open a workspace terminal

1. Compute → **Create compute** → any small all-purpose cluster (e.g. `i3.xlarge`, DBR 14+). Start it.
2. Click the cluster → **Web Terminal** tab → opens a shell inside the cluster VM.

A3. Run the deploy script from the terminal

```
cd /Workspace/Users/<you>/gcp-cost-observability
pip install pyyaml

# Install Databricks CLI if missing
curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh

# Authenticate to the same workspace (uses OAuth in the terminal)
databricks auth login --host https://<your-workspace-url>   # pick a profile name

# Copy + edit config.yml
cp config.example.yml config.yml
vi config.yml                                   # fill in workspace, catalog, region,
emails
                                                # set databricks_profile to the profile
you just created

./scripts/deploy.py --target=prod --run-etl
```

From here, open the dashboard via **Dashboards** in the left nav. Done.

> **Why this is "UI"**: you never leave the workspace. The terminal is a UI feature. No local install required.

Path B — Fully manual UI (no terminal, no CLI)

If you can't run scripts (strict policy / no terminal access), do this. It reproduces what `deploy.py` does, step-by-step.

B1. Create the schema and tables

Open the **SQL Editor** in the workspace. Pick your serverless warehouse. Paste and run:

```sql
-- Replace MAIN and OBSERVABILITY with your catalog + schema
CREATE SCHEMA IF NOT EXISTS MAIN.OBSERVABILITY;

USE CATALOG MAIN;
USE SCHEMA OBSERVABILITY;

CREATE TABLE IF NOT EXISTS daily_cost_summary (
  usage_date              DATE,
  sku_name                STRING,
  billing_origin_product  STRING,
  cloud                   STRING,
  workspace_id            STRING,
  is_serverless           BOOLEAN,
  total_dbus              DECIMAL(38,6),
  list_price_per_dbu      DECIMAL(38,6),
  total_cost_usd          DECIMAL(38,6),
  _etl_updated_at         TIMESTAMP
) USING DELTA PARTITIONED BY (usage_date);

CREATE TABLE IF NOT EXISTS sku_daily_details (
  usage_date              DATE,
  sku_name                STRING,
  billing_origin_product  STRING,
  product_category        STRING,
  is_serverless           BOOLEAN,
  is_photon               BOOLEAN,
  total_dbus              DECIMAL(38,6),
  total_cost_usd          DECIMAL(38,6),
  cost_7d_avg             DECIMAL(38,6),
  cost_30d_avg            DECIMAL(38,6),
  cost_pct_change_7d      DECIMAL(10,2),
  _etl_updated_at         TIMESTAMP
) USING DELTA PARTITIONED BY (usage_date);

CREATE TABLE IF NOT EXISTS gcp_vm_reference_prices (
  node_type_id        STRING,
  gcp_machine_type    STRING,
  vcpus               INT,
  memory_gb           DOUBLE,
  hourly_price_usd    DECIMAL(10,6),
  region              STRING,
  price_source        STRING,
  last_updated        TIMESTAMP
) USING DELTA;

CREATE TABLE IF NOT EXISTS daily_cost_with_vm_estimate (
  usage_date              DATE,
  cluster_id              STRING,
  cluster_name            STRING,
  sku_name                STRING,
```

```
  driver_node_type        STRING,
  worker_node_type        STRING,
  num_workers             BIGINT,
  cluster_uptime_hours    DECIMAL(20,4),
  dbu_cost_usd            DECIMAL(38,6),
  estimated_vm_cost_usd   DECIMAL(38,6),
  total_estimated_cost    DECIMAL(38,6),
  driver_vm_type          STRING,
  worker_vm_type          STRING,
  vm_cost_source          STRING,
  _etl_updated_at         TIMESTAMP
) USING DELTA PARTITIONED BY (usage_date);

CREATE TABLE IF NOT EXISTS alert_config (
  config_id                  INT,
  daily_threshold_dollars    DECIMAL(20,2),
  recipient_emails           STRING,
  is_active                  BOOLEAN,
  threshold_percentages      STRING,
  created_at                 TIMESTAMP,
  updated_at                 TIMESTAMP
) USING DELTA;

CREATE TABLE IF NOT EXISTS alert_history (
  alert_id           BIGINT GENERATED ALWAYS AS IDENTITY,
  alert_date         DATE,
  threshold_pct      INT,
  amount_at_alert    DECIMAL(20,2),
  threshold_amount   DECIMAL(20,2),
  daily_threshold    DECIMAL(20,2),
  recipients         STRING,
  sent_at            TIMESTAMP,
  status             STRING
) USING DELTA;

-- Seed alert_config with your desired threshold + recipients
INSERT INTO alert_config VALUES (
  1,
  1000.00,                                      -- daily threshold $
  'ops@example.com,finance@example.com',        -- recipients (comma-separated)
  true,
  '25,50,75,90,100',                            -- tier percentages
  current_timestamp(),
  current_timestamp()
);
```

B2. Import the notebooks

1. Workspace → **Import** → drop in or point at the 6 files in `src/`:

- `01_setup_schema.py`
- `02_refresh_vm_prices.py`
- `03_daily_cost_etl.py`
- `04_vm_cost_estimation.py`
- `05_alert_checker.py`
- `06_send_alert_email.py`

1. Put them in `/Workspace/Shared/gcp-cost-observability/src/` (or anywhere you want; remember the path).
2. You can skip `01_setup_schema.py` if you ran the SQL in B1 manually — it's the same thing.

B3. Build the "Daily ETL" job

Workflows → **Create Job** → name it `GCP Cost Observability - Daily ETL`. Add **4 tasks** with these settings. For every task: **Compute = Serverless**.

#	Task name	Task type	Notebook	Parameters (key=value)	Depends on
1	`setup_schema`	Notebook	`src/01_setup_schema.py`	`catalog=MAIN` `schema=OBSERVABILITY` `daily_threshold_dollars=1000` `alert_recipient_emails=ops@example.com,finance@example.com` `threshold_percentages=25,50,75,90,100`	—
2	`refresh_vm_prices`	Notebook	`src/02_refresh_vm_prices.py`	`catalog=MAIN` `schema=OBSERVABILITY` `cloud=GCP` `region=us-central1` `gcp_billing_api_key_secret_scope=` `gcp_billing_api_key_secret_name=`	`setup_schema`
3	`daily_cost_etl`	Notebook	`src/03_daily_cost_etl.py`	`catalog=MAIN` `schema=OBSERVABILITY` `cloud=GCP`	`setup_schema`
4	`vm_cost_estimation`	Notebook	`src/04_vm_cost_estimation.py`	`catalog=MAIN` `schema=OBSERVABILITY` `cloud=GCP` `region=us-central1`	`daily_cost_etl`, `refresh_vm_prices`

Schedule: right panel → **Add schedule** → Quartz cron `0 0 6 ?`, timezone `Asia/Kolkata`.

B4. Build the "Hourly Alert" job

Workflows → **Create Job** → `GCP Cost Observability - Hourly Alert Checker`. Add **3 tasks**.

#	Task name	Task type	Notebook	Settings
1	`check_alerts`	Notebook	`src/05_alert_checker.py`	Parameters: `catalog=MAIN`, `schema=OBSERVABILITY`, `notification_destination_id=`
2	`gate_alerts_fired`	If/Else Condition	—	Condition: `{{tasks.check_alerts.values.alerts_sent}} > 0` · Depends on `check_alerts`
3	`send_alert_email`	Notebook	`src/06_send_alert_email.py`	Parameters: `catalog=MAIN`, `schema=OBSERVABILITY` · Depends on `gate_alerts_fired (True)` · **Email notifications on success** = your recipient list

Schedule: Quartz cron `0 0 * ?`, timezone `Asia/Kolkata`.

B5. Import the Lakeview dashboard

1. First, render `cost_observability_dashboard.lvdash.json.template` locally or in the SQL editor: replace `{{CATALOG_SCHEMA}}` with your fully-qualified schema (e.g. `main.observability`) and `{{DEFAULT_MONTH_LABEL}}` with the current month label (e.g. `Apr 2026`). 8 occurrences of each.
2. Save the result as `cost_observability_dashboard.lvdash.json`.
3. In the workspace: **Dashboards** → **Create** → **Import File** → upload the rendered `.lvdash.json`.
4. Open the dashboard → verify widgets render with data → click **Publish** to share.

B6. Run the initial ETL

Workflows → your `GCP Cost Observability - Daily ETL` job → **Run now**. Takes 2–6 minutes.

Then open the dashboard — you should see today's data.

Changing config later (both paths)

Path A: edit `config.yml` in the Git Folder, re-run `./scripts/deploy.py --target=prod` from the web terminal.

Path B: everything is in the workspace UI:

Change	Where
Threshold	Jobs → Daily ETL → `setup_schema` task → edit `daily_threshold_dollars` param. Also `UPDATE alert_config SET daily_threshold_dollars = X WHERE config_id = 1` in SQL editor
Recipients (actual email)	Jobs → Hourly Alert → `send_alert_email` task → **Email notifications** → edit list
Recipients (dashboard display only)	`UPDATE alert_config SET recipient_emails = '...' WHERE config_id = 1`
Alert tiers	`UPDATE alert_config SET threshold_percentages = '50,100' WHERE config_id = 1`
Region	Jobs → Daily ETL → `refresh_vm_prices` + `vm_cost_estimation` tasks → edit `region` param
Schedule time / cron / timezone	Jobs → Schedule panel → edit cron + timezone

---

Troubleshooting

Symptom	Fix
Web terminal button missing	Workspace admin may have disabled it. Use Path B, or ask admin to enable Web Terminal
`databricks auth login` fails in terminal	Terminal might need `export DATABRICKS_HOST=<url>` + token-based auth (`databricks auth login --host $DATABRICKS_HOST`)
Dashboard import rejects the JSON	You forgot to substitute `{{CATALOG_SCHEMA}}` or `{{DEFAULT_MONTH_LABEL}}`. Search/replace both and retry
`alert_config` is empty	You skipped the `INSERT INTO alert_config` in B1. Run it now
Job tasks can't find notebook	Path in the job's task spec doesn't match where you imported the notebook. Edit the task's **Path** field
Condition task syntax unfamiliar	In the "If/Else" task, the expression syntax is `{{tasks.<task_key>.values.<value_key>}} <op> <literal>`

Uninstall (UI)

- Workflows → delete both jobs
- Dashboards → delete the dashboard
- SQL editor: `DROP SCHEMA <catalog>.<schema> CASCADE`
- Workspace → delete the imported notebooks folder

That's it. Nothing outside the workspace was touched.